Compiler Design
Assignment 1
Lexical Analyzer for C^-

Mai Mahmoud Sayed
ID: 900213311

Spring 2026

Contents

1 Introduction **2**

2 Language Specification **2**
2.1 Keywords 2
2.2 Special Symbols 2
2.3 Identifier Definition 2
2.4 Number Definition 2
2.5 Whitespace and Comments 3

3 Design Decisions **3**
3.1 Rule Ordering and Longest-Match 3
3.2 Position Tracking 3
3.3 Comment Handling 3

4 Error Handling **3**

5 Build Instructions **4**
5.1 Requirements 4
5.2 Build and Run 4
5.3 Output Format 4

6 Testing **4**
6.1 Test 1 – Valid C^- Program (`test1_valid.cm`) 5
6.2 Test 2 – Error Cases (`test2_errors.cm`) 6

7 Conclusion **8**

1 Introduction

This project implements a lexical scanner for the C^- programming language using GNU Flex. The scanner reads source code and identifies tokens: keywords, identifiers, numeric literals, operators, and punctuation. It also detects four categories of lexical error and reports each with a precise line number and column position.

Token definitions follow Appendix A.1 of the textbook, with modified regular expressions for identifiers and numbers as specified in the assignment.

2 Language Specification

2.1 Keywords

The following six reserved words must appear in lowercase and cannot be used as identifiers:

`else`	`if`	`int`	`return`	`void`	`while`

2.2 Special Symbols

Token	Lexeme	Token	Lexeme	Token	Lexeme
PLUS	+	MINUS	-	TIMES	*
DIVIDE	/	LT	<	LTE	<=
GT	>	GTE	>=	EQ	==
NEQ	!=	ASSIGN	=	SEMI	;
COMMA	,	LPAREN	(	RPAREN	)
LBRACKET	[	RBRACKET	]	LBRACE	{
RBRACE	}				

2.3 Identifier Definition

$$ID = \texttt{letter}\,(\texttt{letter} \mid \texttt{digit})^* \left([\texttt{.\#\$_}]\,(\texttt{letter} \mid \texttt{digit})^+ \right)^?$$

The base is one letter followed by zero or more alphanumeric characters. An optional suffix is exactly one special character from {`.` `#` `$` `_`} followed by one or more alphanumeric characters. At most one suffix is allowed. Upper- and lowercase letters are distinct.

Valid: `x`, `count#2`, `var.next`, `my$data`, `abc_xyz`

Invalid: `a#`, `b.`, `c$`, `d_` (special character at end with nothing following)

2.4 Number Definition

$$NUM = \left(\texttt{digit}^+ \mid \texttt{digit}^+\texttt{.}\,\texttt{digit}^*\right) \left([\texttt{Ee}]\,[+-]^?\,\texttt{digit}^+ \right)^?$$

Covers integers, decimal literals (e.g. `3.` and `3.14`), and an optional scientific-notation exponent. Numbers may **not** begin with `.`; a leading dot is reported as an invalid character.

Valid: `42`, `3.14`, `3.`, `6.022E23`, `1.5e-10`, `100E+3`

Invalid: `23e+`, `5.0E-` (exponent sign not followed by digits)

2.5 Whitespace and Comments

Blanks, tabs, and newlines are ignored except as separators. Comments are delimited by `/* ...*/`, may span multiple lines, and cannot be nested.

3 Design Decisions

3.1 Rule Ordering and Longest-Match

Flex selects the rule that matches the most characters (longest-match); ties go to the first listed rule. This drives three key ordering decisions:

- **Keywords before ID.** `int` matches both patterns at the same length; first-rule priority correctly classifies it as a keyword. A longer token like `integer` always wins by longest-match.
- **Multi-character operators before single-character prefixes.** `<=`, `>=`, `==`, `!=` are listed before `<`, `>`, `=` to ensure the longer token is always preferred.
- **Error rules after valid rules.** The wrong-ID and wrong-NUM patterns are placed after their valid counterparts. Longest-match automatically selects the error rule only when no valid match is longer, eliminating the need for trailing-context operators.

3.2 Position Tracking

The macro `YY_USER_ACTION` is defined in the prologue and invoked automatically by Flex before every rule action. It snapshots `tok_line` and `tok_col`, then advances `col_num` by `yyleng`. The newline rule increments `line_num` and resets `col_num` to 1.

3.3 Comment Handling

When `/*` is matched, the scanner saves the opening position and calls `input()` in a loop until `*/` or EOF is reached. Because `input()` bypasses `YY_USER_ACTION`, position counters are updated manually inside the loop. On EOF, an error message is printed and `yyterminate()` halts scanning.

4 Error Handling

The scanner handles four error categories, continuing after types 2–4 to maximise diagnostic coverage in a single pass.

#	Error	Description and Mechanism
1	EOF in unclosed comment	A `while` loop calls `input()` after `/*` is matched. On EOF, `yyterminate()` stops scanning and reports the opening location.
2	Invalid character	The catch-all dot rule `.`, placed last, matches any character not accepted by any earlier rule and reports its ASCII value.
3	Wrong identifier	The wrong-ID pattern fires when a special character appears at the end of an identifier with no alphanumeric character following. Longest-match ensures the valid rule wins whenever a legal suffix is present.
4	Wrong number	The wrong-NUM pattern fires when an exponent sign is not immediately followed by digits. The entire malformed prefix (e.g. `23e+`) is consumed and reported as one error.

5 Build Instructions

5.1 Requirements

- **flex** (GNU flex 2.6+)
- **GCC** (or any C99-compatible compiler)
- **GNU make**

5.2 Build and Run

```
make                        # build using the Makefile
./scanner input_file.cm     # scan a file
./scanner                   # read from stdin
make test                   # run both test cases
```

5.3 Output Format

Tokens are printed to **stdout** and errors to **stderr**:

```
Line   L, Col   C | Token: TOKEN_NAME   | Lexeme: "lexeme"
LEXICAL ERROR [Line L, Col C]: <descriptive message>
```

6 Testing

Two test files were used to verify the scanner. The full outputs are shown in the attached screenshots.

6.1 Test 1 – Valid C⁻ Program (`test1_valid.cm`)

This file exercises all six keywords, all operators, identifiers with all four special characters (`#`, `.`, `$`, `_`), and number literals including integers, trailing-decimal floats (`3.`), and scientific notation (`6.022E23`, `1.5e-10`). One Error 2 case is included: the leading-decimal form `.14`, which correctly produces an invalid-character message followed by `NUM "14"`.

```
Scanning: test1_valid.cm

Position                  Token          Lexeme
------------------------ -------------- --------------------------
Line   5, Col   1 | Token: INT          | Lexeme: "int"
Line   5, Col   5 | Token: ID           | Lexeme: "gcd"
Line   5, Col   8 | Token: LPAREN       | Lexeme: "("
Line   5, Col   9 | Token: INT          | Lexeme: "int"
Line   5, Col  13 | Token: ID           | Lexeme: "u"
Line   5, Col  14 | Token: COMMA        | Lexeme: ","
Line   5, Col  16 | Token: INT          | Lexeme: "int"
Line   5, Col  20 | Token: ID           | Lexeme: "v"
Line   5, Col  21 | Token: RPAREN       | Lexeme: ")"
Line   5, Col  23 | Token: LBRACE       | Lexeme: "{"
Line   6, Col   5 | Token: IF           | Lexeme: "if"
Line   6, Col   8 | Token: LPAREN       | Lexeme: "("
Line   6, Col   9 | Token: ID           | Lexeme: "v"
Line   6, Col  11 | Token: EQ           | Lexeme: "=="
Line   6, Col  14 | Token: NUM          | Lexeme: "0"
Line   6, Col  15 | Token: RPAREN       | Lexeme: ")"
Line   6, Col  17 | Token: RETURN       | Lexeme: "return"
Line   6, Col  24 | Token: ID           | Lexeme: "u"
Line   6, Col  25 | Token: SEMI         | Lexeme: ";"
Line   7, Col   5 | Token: ELSE         | Lexeme: "else"
Line   7, Col  10 | Token: RETURN       | Lexeme: "return"
Line   7, Col  17 | Token: ID           | Lexeme: "gcd"
Line   7, Col  20 | Token: LPAREN       | Lexeme: "("
Line   7, Col  21 | Token: ID           | Lexeme: "v"
Line   7, Col  22 | Token: COMMA        | Lexeme: ","
Line   7, Col  24 | Token: ID           | Lexeme: "u"
Line   7, Col  26 | Token: MINUS        | Lexeme: "-"
Line   7, Col  28 | Token: LPAREN       | Lexeme: "("
```

Figure 1: Test 1 scanner output – part 1

```
Line  32, Col   5 | Token: ID          | Lexeme: "nums"
Line  32, Col   9 | Token: LBRACKET    | Lexeme: "["
Line  32, Col  10 | Token: NUM         | Lexeme: "10"
Line  32, Col  12 | Token: RBRACKET    | Lexeme: "]"
Line  32, Col  13 | Token: SEMI        | Lexeme: ";"
Line  33, Col   1 | Token: ID          | Lexeme: "result_val"
Line  33, Col  12 | Token: ASSIGN      | Lexeme: "="
Line  33, Col  14 | Token: NUM         | Lexeme: "3.14"
Line  33, Col  18 | Token: SEMI        | Lexeme: ";"
Line  34, Col   1 | Token: ID          | Lexeme: "result_val"
Line  34, Col  12 | Token: ASSIGN      | Lexeme: "="
Line  34, Col  14 | Token: NUM         | Lexeme: "3."
Line  34, Col  16 | Token: SEMI        | Lexeme: ";"
Line  35, Col   1 | Token: ID          | Lexeme: "result_val"
Line  35, Col  12 | Token: ASSIGN      | Lexeme: "="
LEXICAL ERROR [Line 35, Col 14]: Invalid character '.' (ASCII 46) -- not in the language alphabet
Line  35, Col  15 | Token: NUM         | Lexeme: "14"
Line  35, Col  17 | Token: SEMI        | Lexeme: ";"
Line  36, Col   1 | Token: ID          | Lexeme: "result_val"
Line  36, Col  12 | Token: ASSIGN      | Lexeme: "="
Line  36, Col  14 | Token: NUM         | Lexeme: "42"
Line  36, Col  16 | Token: SEMI        | Lexeme: ";"
Line  37, Col   1 | Token: ID          | Lexeme: "result_val"
Line  37, Col  12 | Token: ASSIGN      | Lexeme: "="
Line  37, Col  14 | Token: NUM         | Lexeme: "6.022E23"
Line  37, Col  22 | Token: SEMI        | Lexeme: ";"
Line  38, Col   1 | Token: ID          | Lexeme: "result_val"
Line  38, Col  12 | Token: ASSIGN      | Lexeme: "="
Line  38, Col  14 | Token: NUM         | Lexeme: "1.5e-10"
Line  38, Col  21 | Token: SEMI        | Lexeme: ";"
Line  39, Col   1 | Token: ID          | Lexeme: "result_val"
Line  39, Col  12 | Token: ASSIGN      | Lexeme: "="
```

Figure 2: Test 1 scanner output – part 2

6.2 Test 2 – Error Cases (`test2_errors.cm`)

This file deliberately triggers all four error categories:

- **Error 3** – Five bad identifiers (`a#`, `b.`, `c$`, `d_`, `e#`); scanning resumes at the following semicolon in each case.
- **Error 4** – Three malformed numbers (`23e+`, `5.0E-`, `100e+`); for `100e+*;` the scanner correctly emits `TIMES` and `SEMI` after the error.
- **Error 2** – Three invalid characters (`@`, `^`, `?`); surrounding valid tokens are unaffected.
- **Error 1** – An unclosed comment at end-of-file; scanning halts after reporting the opening position.

```
Position                 Token          Lexeme
------------------------ -------------- --------------------------
Line   4, Col   1 | Token: INT          | Lexeme: "int"
LEXICAL ERROR [Line 4, Col 5]: Invalid identifier "a#" -- special character '#' must be followed by a letter or digit
Line   4, Col   7 | Token: SEMI         | Lexeme: ";"
Line   5, Col   1 | Token: INT          | Lexeme: "int"
LEXICAL ERROR [Line 5, Col 5]: Invalid identifier "b." -- special character '.' must be followed by a letter or digit
Line   5, Col   7 | Token: SEMI         | Lexeme: ";"
Line   6, Col   1 | Token: INT          | Lexeme: "int"
LEXICAL ERROR [Line 6, Col 5]: Invalid identifier "c$" -- special character '$' must be followed by a letter or digit
Line   6, Col   7 | Token: SEMI         | Lexeme: ";"
Line   7, Col   1 | Token: INT          | Lexeme: "int"
LEXICAL ERROR [Line 7, Col 5]: Invalid identifier "d_" -- special character '_' must be followed by a letter or digit
Line   7, Col   7 | Token: SEMI         | Lexeme: ";"
Line   8, Col   1 | Token: INT          | Lexeme: "int"
LEXICAL ERROR [Line 8, Col 5]: Invalid identifier "e#" -- special character '#' must be followed by a letter or digit
Line   8, Col   9 | Token: SEMI         | Lexeme: ";"
Line  11, Col   1 | Token: INT          | Lexeme: "int"
Line  11, Col   5 | Token: ID           | Lexeme: "x"
Line  11, Col   6 | Token: SEMI         | Lexeme: ";"
Line  12, Col   1 | Token: ID           | Lexeme: "x"
Line  12, Col   3 | Token: ASSIGN       | Lexeme: "="
LEXICAL ERROR [Line 12, Col 5]: Invalid number "23e+" -- exponent sign must be immediately followed by one or more digits
Line  12, Col   9 | Token: SEMI         | Lexeme: ";"
Line  13, Col   1 | Token: ID           | Lexeme: "x"
Line  13, Col   3 | Token: ASSIGN       | Lexeme: "="
LEXICAL ERROR [Line 13, Col 5]: Invalid number "5.0E-" -- exponent sign must be immediately followed by one or more digits
Line  13, Col  10 | Token: SEMI         | Lexeme: ";"
Line  14, Col   1 | Token: ID           | Lexeme: "x"
Line  14, Col   3 | Token: ASSIGN       | Lexeme: "="
LEXICAL ERROR [Line 14, Col 5]: Invalid number "100e+" -- exponent sign must be immediately followed by one or more digits
```

Figure 3: Test 2 error output – part 1

```
Line  13, Col  10 | Token: SEMI         | Lexeme: ";"
Line  14, Col   1 | Token: ID           | Lexeme: "x"
Line  14, Col   3 | Token: ASSIGN       | Lexeme: "="
LEXICAL ERROR [Line 14, Col 5]: Invalid number "100e+" -- exponent sign must be immediately followed by one or more digits
Line  14, Col   3 | Token: ASSIGN       | Lexeme: "="
LEXICAL ERROR [Line 14, Col 5]: Invalid number "100e+" -- exponent sign must be immediately followed by one or more digits
LEXICAL ERROR [Line 14, Col 5]: Invalid number "100e+" -- exponent sign must be immediately followed by one or more digits
Line  14, Col  10 | Token: TIMES        | Lexeme: "*"
Line  14, Col  11 | Token: SEMI         | Lexeme: ";"
Line  17, Col   1 | Token: ID           | Lexeme: "x"
Line  17, Col   3 | Token: ASSIGN       | Lexeme: "="
Line  17, Col   5 | Token: NUM          | Lexeme: "3"
LEXICAL ERROR [Line 17, Col 7]: Invalid character '@' (ASCII 64) -- not in the language alphabet
Line  17, Col   9 | Token: NUM          | Lexeme: "4"
Line  17, Col  10 | Token: SEMI         | Lexeme: ";"
Line  18, Col   1 | Token: ID           | Lexeme: "x"
Line  14, Col  11 | Token: SEMI         | Lexeme: ";"
Line  17, Col   1 | Token: ID           | Lexeme: "x"
Line  17, Col   3 | Token: ASSIGN       | Lexeme: "="
Line  17, Col   5 | Token: NUM          | Lexeme: "3"
LEXICAL ERROR [Line 17, Col 7]: Invalid character '@' (ASCII 64) -- not in the language alphabet
Line  17, Col   9 | Token: NUM          | Lexeme: "4"
Line  17, Col  10 | Token: SEMI         | Lexeme: ";"
Line  18, Col   1 | Token: ID           | Lexeme: "x"
Line  17, Col   3 | Token: ASSIGN       | Lexeme: "="
Line  17, Col   5 | Token: NUM          | Lexeme: "3"
LEXICAL ERROR [Line 17, Col 7]: Invalid character '@' (ASCII 64) -- not in the language alphabet
Line  17, Col   9 | Token: NUM          | Lexeme: "4"
Line  17, Col  10 | Token: SEMI         | Lexeme: ";"
Line  18, Col   1 | Token: ID           | Lexeme: "x"
```

Figure 4: Test 2 error output – part 2

7 Conclusion

The Flex specification correctly tokenizes all legal C^- constructs and detects all four required lexical error categories with precise diagnostics. The key design insight is that wrong-identifier and wrong-number errors are handled automatically by Flex's longest-match rule, without trailing-context operators, keeping the specification concise and unambiguous.